

POLITECNICO
MILANO 1863

Bioinformatics Algorithms

Sequences and sequence alignment (part 1)

Rosario M. Piro
(based on slides by Giulio Pavesi)

Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB)
Politecnico di Milano

Sequences

- Information is encoded in living cells (mostly) in the form of “sequences”:
  - DNA sequences (nucleotide sequences; nucleic acids; double-stranded; ACGT)
  - RNA sequences (nucleotide sequences; nucleic acids; single-stranded; ACGU)
  - protein sequences (amino acid sequences; polypeptides; 20 “letters”)

- This is one of the main reasons of the success of applying information science to the study of biological molecules

- Biological function largely depends on the 3D structure of large molecules (e.g., protein–protein interactions, DNA binding of transcription factors, etc.), but many characteristics or functions can also be inferred from the 1D structure (the sequences)
  - Gene activity (gene “expression”) can be measured by sequencing RNA molecules
  - Alternative splicing can be identified by sequencing RNA molecules
  - 3D structure can (often) be predicted from the sequence (proteins, RNA moleucles)
  - The genome sequence can be searched for sequence patterns to identify new genes
  - ...

Alphabets and strings

- Let Σ be a finite set of symbols (=characters), called the alphabet.
  No assumption is made about the nature of the symbols.
  - E.g., for DNA sequences, we usually consider $\Sigma = \{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}\}$ (and for RNA U instead of T), but there may actually be additional symbols (e.g., N = any nucleotide; R = purine = A or G; Y = pyrimidine = C or $\mathrm{T/U}$; ...)

- A string (or word) over Σ is any finite sequence of symbols from Σ.
  For example, if $\Sigma = \{0, 1\}$, then 01011 is a string over Σ.
  - Sequence: a set of elements ordered so that they can be labeled/enumerated with positive integers; repetitions are allowed
    - $S = s_1 s_2 s_3 \ldots s_n$
    - In biology: usually 1-based numbering (i.e., starting from 1, not from 0)
    - Orientation of DNA/RNA sequences: from $5'$ to $3'$ (see first lecture)
    - Orientation of protein sequences: from N-terminus to C-terminus (see first lecture)
    - Double-stranded DNA is usually represented by a single strand (the other strand is its "reverse complement")

- The length of a string S is the number of symbols in S (the length of the sequence) and can be any non-negative integer; it is often denoted as $|S|$

- The empty string is the unique string over Σ of length 0, and is denoted ε

Protein sequences

- Protein sequences are usually represented by strings over a 20-letter alphabet: $\Sigma = \{A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y\}$
- These 20 letters are usually sufficient, but:
  - The "mysterious" 21st amino acid (selenocysteine) is denoted by "U"
  - "Unknown" amino acids in a sequence are denoted by X
- Orientation of protein sequences: from N-terminus to C-terminus
  - A protein sequence always starts with an "M" (methionine; encoded by the start codon "AUG")
  - The stop codons ("UAA", "UAG" and "UGA") do not encode for any amino acid but terminate translation (sometimes they are represented as translating to "*", but the asterisk is not considered to be part of Σ because nothing is added to the protein molecule)

More about strings

- The set of all possible strings over Σ of length n is denoted Σ^n
  - For $\Sigma = \{0, 1\}$ we have $\Sigma^2 = \{00, 01, 10, 11\}$
- Note that $\Sigma^0 = \{\varepsilon\}$ for any alphabet Σ
- The (infinite) set of all strings over Σ of any length is the Kleene closure of Σ ("zero or more repetitions" of arbitrary elements of Σ) and is denoted Σ^*, thus:

$$\Sigma^* = \bigcup_{n \in \mathbb{N} \cup \{0\}} \Sigma^n$$

  - For $\Sigma = \{0, 1\}$ we have $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, \ldots\}$
  - The (infinite) set of all non-empty strings over Σ is denoted as Σ^+

Concatenations and substrings

- For any two strings s and t in Σ^*, their concatenation is defined as the sequence of symbols in s followed by the sequence of characters in t, and is denoted st
  - Let $\Sigma = \{a, b, \ldots, z\}$, s="honey", t="bee"
    Then, st="honeybee", ts="beehoney"
  - For any string s we have $\varepsilon s = s\varepsilon = s$

- A string s is a substring (or factor) of t if there exist (possibly empty) strings u and v such that $t = usv$
  Special cases:
  - Prefix: $u = \varepsilon$, i.e., $t = sv$
    - *Proper* prefix: $u = \varepsilon$ but $v \neq \varepsilon$
  - Suffix: $v = \varepsilon$, i.e., $t = us$
    - *Proper* suffix: $v = \varepsilon$ but $u \neq \varepsilon$

  How many substrings does t have?
  - 1 substring of length $n = |t|$; 2 substrings of length $n-1$; ...; n substrings of length 1
  - Plus 1 substring of length 0 (empty string ε)
  - Thus:

$$1 + \sum_{i=1}^{n} i = 1 + \frac{n(n+1)}{2}$$

Reverse strings, palindromes and rotations

- The reverse of a string is a string with the same symbols but in reverse order
  - For example, if s = "abc" (with a,b,c $\in \Sigma$), then the reverse of s is "cba"

- A string that is the reverse of itself (e.g., s = "omordotuanuoraoarounautodromo") is called a palindrome
  - The empty string and all strings of length 1 are palindromes
  - Note: while some wordplays consider spaces as "moveable" because they aren't spoken, we consider as palindromes only strings where the order of *all* symbols (including spaces) is *precisely* reversed, e.g., we would consider "stressed desserts" to be a palindrome, but not "a nut for a jar of tuna")
  - **Difference for DNA/RNA**: a palindromic DNA/RNA sequence is identical to its **reverse complement** (reading both strands 5'$\rightarrow$3')!
    Example: "GATC" is palindromic "CAAC" is not!

- A string $s = uv$ (with $u,v \in \Sigma^*$) is said to be a rotation of t if $t = vu$
  - For example, if $\Sigma = \{0,1\}$, the string 0011001 is a rotation of 0100110, where u = 00110 and v = 01
  - As another example, the string "abc" has three different rotations:
    - itself (with u="abc", v=ε)
    - bca (with u="bc", v="a"), and
    - cab (with u="c", v="ab")
  - The number of possible rotations of a string is equal to its length!

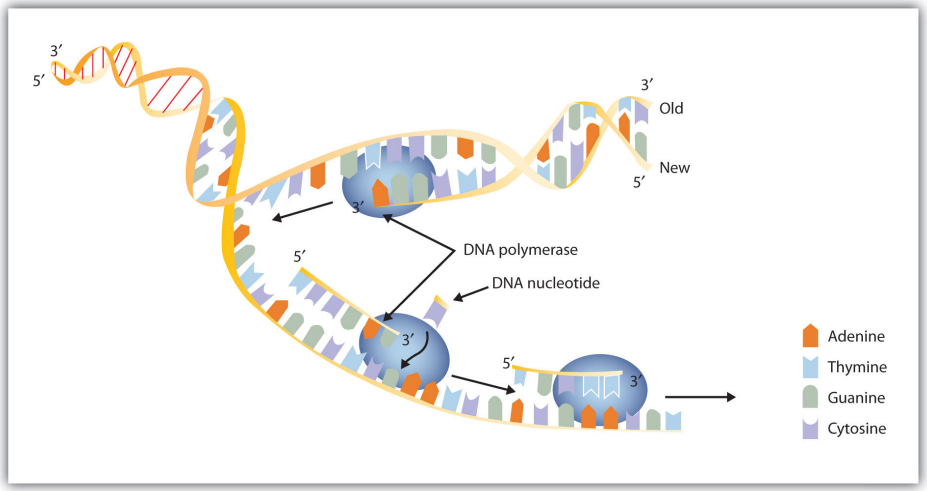
Comparison of biological sequences

- An important goal is to define suitable methods for the comparison of sequences:
  - To assess or quantify “how different” or “how similar” two or more sequences are
  - To identify similar sequences (with similar functions?) in databases
  - To identify the genomic locations of NGS reads
  - ...
- We need to define suitable measures of distance or similarity, as well as efficient algorithms to compute them
- The measures should reflect as much as possible the different ways in which the sequences can be similar or different, and the processes or factors underlying the similarities and differences, for example:
  - Evolution of the sequences: what does it mean that we “share 50% of our genes with bananas”? More here: https://medium.com/emergent-phenomena/we-share-50-of-genes-with-bananas-and-50-from-either-parent-is-one-of-our-parents-a-banana-495558086405
  - Errors in data production (e.g., sequencing errors)
  - Errors made my the cells, e.g., during DNA replication; more here: https://www.nature.com/scitable/topicpage/dna-replication-and-causes-of-mutation-409/

Technical errors in biological sequences

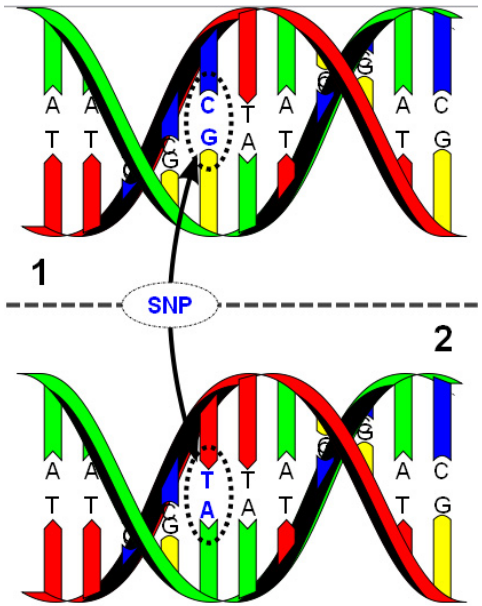
- Sequencing error: the sequencing machine identifies a wrong base/nucleotide in a sequence (e.g., “ACGCGGTGAAC” instead of “ACGCGTTGAAC”)
- Error rates change according to the sequencing platform: for Illumina sequencers (most widely used) <0.1%
  - That is, less than one sequencing error every 1000 base pairs
  - But: in 100 million reads of 200 bp length each you would still find up to 20 million sequencing errors ...
- Such errors cannot be ignored by algorithms employed in the analysis of the data

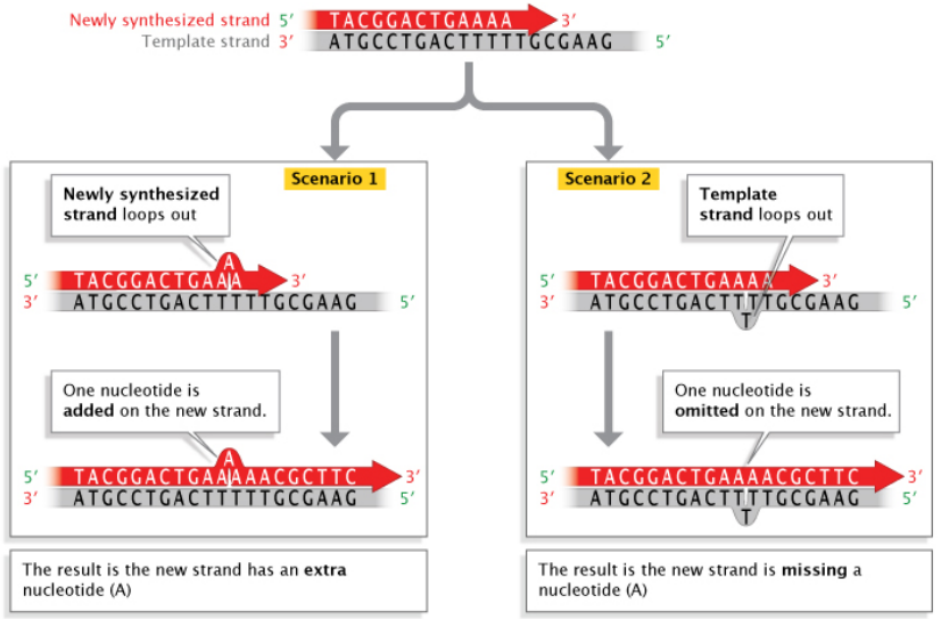
Biological errors in biological sequences



- Living cells replicate their DNA and replication errors are *naturally* part of this process
  - Although cells possess very efficient (and beautiful!) error-correcting mechanisms acting at different points of DNA replication, some errors escape ("mutations") and are inherited by daughter cells
  - The "error rate" changes according to the organism and the different parts of the genome, roughly ranging from one every 100 million base pairs to (in some cases) one every thousands of base pairs
- What types of error does the cell make in replicating its DNA?

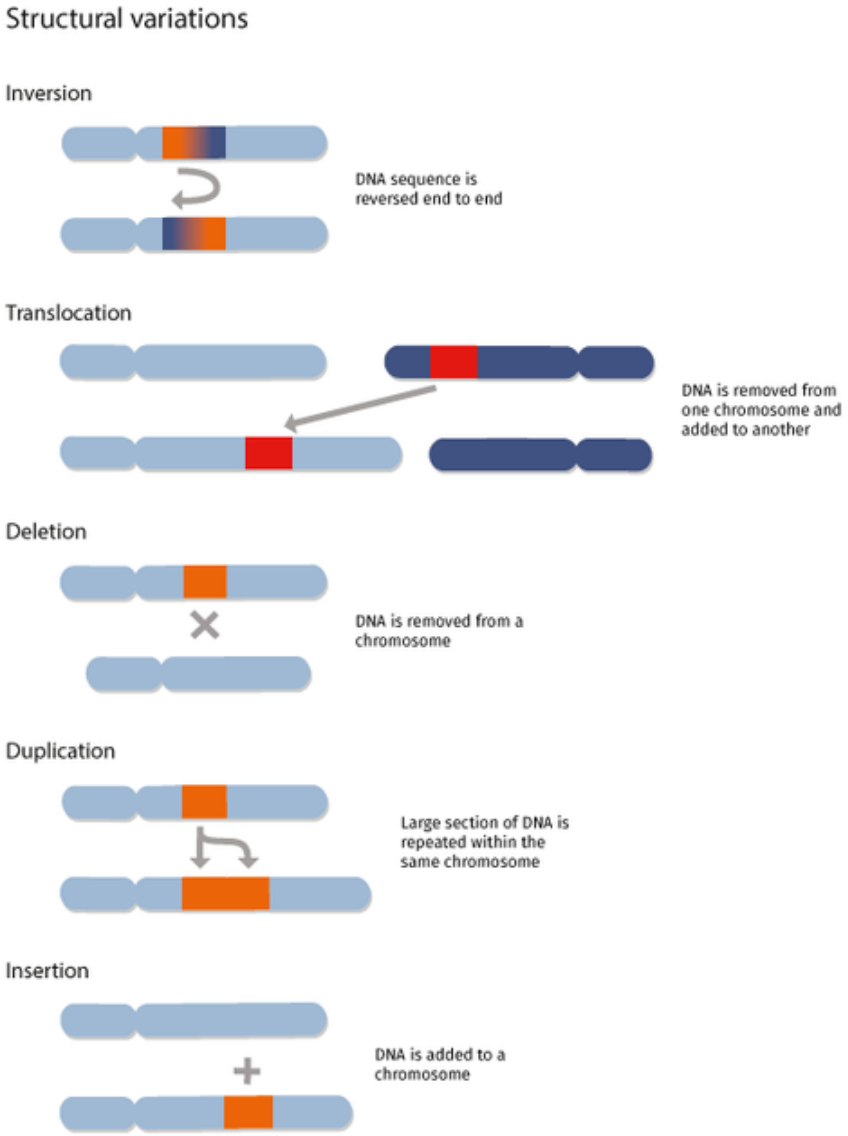
Biological errors in biological sequences





- Base substitutions (“single nucleotide polymorphisms/variants”; SNPs, SNVs)
  - Like for sequencing errors, the original base pair is erroneously replaced by another one. Depending on which base is altered/mutated, this may lead to disease
  - Difference to sequencing errors: the latter affects only one read; mutations affect many (or all) reads which cover the mutated genomic location
- Insertions/deletions of small DNA fragments (“indels”) (even of a single base pair)

Biological errors in biological sequences



(from: `https://www.garvan.org.au/`)

- Structural variants
  - Involve regions of DNA of about 1 kb and larger in size
  - Mentioned here for completeness (less important for the algorithms we discuss in this course)
  - Inversion of a genomic region (orientation inverted)
  - Translocation of a genomic region
    - ★ "*inter*-chromosomal": from one chromosome to another
    - ★ "*intra*-chromosomal": within a chromosome
  - (Large-scale) insertions/deletions (as opposed to indels)
  - Duplications
    - ★ "tandem duplication": adjacent to original sequence
    - ★ or region duplicated elswhere

Comparing strings: Hamming distance

- The **Hamming distance** between two strings of *equal length* is the *number of positions* at which the corresponding symbols are different

- The Hamming distance between ...
  - “karolin” and “kathrin” is 3
  - “karolin” and “kerstin” is 3
  - 1011101 and 1001001 is 2
  - 2173896 and 2233796 is 3

- Problem: what about the following two 8 bp sequences:

  ```
  ACGTACGT
  CGTACGTA
  ```

  Their Hamming distance is 8 (symbols are different in *all* positions); but are these sequences really so dissimilar???

- With the Hamming distance we can formalize “substitutions” in otherwise identical biological sequences – or simply sequencing errors in which the wrong base pairs were identified
  - Not a good measure for (dis)similarity in general!

Comparing strings: the edit distance

- The **edit distance** is a way of quantifying how dissimilar two strings (also of different length) are to one another by counting the *minimum number of operations required to transform one string into the other*

- Different definitions of an edit distance use different sets of string operations. In the **Levenshtein distance** the allowed operations are
  - removal
  - insertion
  - substitution

  of a character in the string. Being the most common metric, the term "Levenshtein distance" is often used interchangeably with "edit distance" (as we will also do in this course)

- The edit distance was invented before bioinformatics but captures nicely "biological errors" in sequences
  - substition, insertion or deletion of a nucleotide

- Interesting also for other purposes, e.g., for spelling correction:
  - A user erroneously typed "graffe"; what did the user most likely mean? "graf", "graft", "grail" or "giraffe"?

Comparing strings: the edit distance

Possible mathematical definition (https://en.wikipedia.org/wiki/Levenshtein_distance)

$$\operatorname{lev}(a,b) = \begin{cases} |a| & \text{if } |b| = 0, \\ |b| & \text{if } |a| = 0, \\ \operatorname{lev}\left(\operatorname{tail}(a), \operatorname{tail}(b)\right) & \text{if } a[0] = b[0] \\ 1 + \min \begin{cases} \operatorname{lev}\left(\operatorname{tail}(a), b\right) \\ \operatorname{lev}\left(a, \operatorname{tail}(b)\right) \\ \operatorname{lev}\left(\operatorname{tail}(a), \operatorname{tail}(b)\right) \end{cases} & \text{otherwise,} \end{cases}$$

(Note: we will see an alternative definition later!)

Recursive definition, where

- *tail(x)* is the string *x* without its first character
- $x[n]$ is the *n*-th character of string *x* (here: 0-based)
- In the minimum term, asuming we want to convert *a* into *b*:
  - First element corresponds to a deletion/removal[1] of a character from *a*
  - Second element corresponds to an insertion[1] of a character into *a*
  - Third element correpsonds to a replacement/substitution
- Example: the Levenshtein distance between “kitten” and “sitting” is 3; there is no way to do it with fewer than 3 edits:
  1. kitten $\rightarrow$ sitten (substitution of “s” for “k”)
  2. sitten $\rightarrow$ sittin (substitution of “i” for “e”)
  3. sittin $\rightarrow$ sitting (insertion of “g” at the end)

[1] For instead coverting *b* into *a*, the 1st element is an insertion into *b* and the 2nd a removal from *b*!

Comparing strings: the edit distance

- From the mathematical definition, it is evident that computing the edit (Levenshtein) distance between two strings implies solving a minimization problem: find among all the possible alternatives, the one that requires the minimum number of operations
  - The number of solutions (sequences of edit operations) is infinite
  - In practice, we would not need to test all of them, because any solution with more than $max(|a|, |b|)$ edit operations can be excluded; so we could discard partial solutions as soon as we exceed this number of operations

- Representing operations: "INTENTION" vs. "EXECUTION"
  - Operations:

    INTENTION
    ~~**I**~~**NTENTION**
    ETENTION
    EXENTION
    EXECTION
    EXECUTION

    (where ~~I~~ is a deletion, E a substitution, and U an insertion)

- Compact representation:

  INTEN-TION
  -EXECUTION

- This is a **sequence alignment**[2]:
  - Write the two strings one below the other and highlight where edit operations took place
    - ★ *gaps* ("−") to indicate deletions / insertions (symmetry: insertion in string A = deletion in string B)
  - Intuitive overview of differences

[2] Ambiguity of the term: we perform a sequence alignment to obtain a sequence alignment

Alignment for minimum edit distance

Can we exploit the edit distance to fully "align" two sequences with respect to their similarities and differences, i.e., to determine their best "alignment"?[3]

- Denote as $E(a, b)$ the edit distance between two strings a and b
- For every string a we have $E(a, \varepsilon) = |a|$, that is
  - The edit distance between a and the empty string quals the length of a
  - Also: $E(\varepsilon, \varepsilon) = 0$
- Let $a = a_1 a_2 \dots a_n$ be a string of length $|a| = n$, and $b = b_1 b_2 \dots b_m$ a string of length $|b| = m$
  - We denote as $a(i)$ the prefix of length $i \leq n$ of string a: $a(i) = a_1 a_2 \dots a_i$
  - Likewise, $b(j)$ is the prefix of length $j \leq m$ of string b: $b(j) = b_1 b_2 \dots b_j$
- To compute the edit distance $E(a, b)$ between a and b, let's first assume that for given i and j we (somehow) already computed
  - $E\big(a(i-1), b(j-1)\big)$ = edit dist. for the prefixes of $a(i)$ and $b(j)$ both with length -1
  - $E\big(a(i), b(j-1)\big)$ = edit dist. for $a(i)$ and the prefix of $b(j)$ with length -1
  - $E\big(a(i-1), b(j)\big)$ = edit dist. for $b(j)$ and the prefix of $a(i)$ with length -1
- Dynamic programming: solve a complex problem by breaking it down into simpler sub-problems in a recursive manner

[3] Here, we will develop an alternative definition of the edit distance, not going iteratively from the last letters to the first (iteratively using *tail*), but instead from the first to the last (using prefixes)!

Alignment for minimum edit distance

- To compute the edit distance $E(a,b)$ between a and b, let's first assume that for given i and j we (somehow) already computed
  - $E\big(a(i-1),b(j-1)\big)$ = edit dist. for the prefixes of $a(i)$ and $b(j)$ with length -1
  - $E\big(a(i),b(j-1)\big)$ = edit dist. for $a(i)$ and the prefix of $b(j)$ with length -1
  - $E\big(a(i-1),b(j)\big)$ = edit dist. for $b(j)$ and the prefix of $a(i)$ with length -1

- Then, it can be proven that the following is true:

$$E\big(a(i),b(j)\big) = min \begin{cases} 1 + E\big(a(i),b(j-1)\big) \\ 1 + E\big(a(i-1),b(j)\big) \\ \begin{cases} E\big(a(i-1),b(j-1)\big) & \text{if } a_i = b_j \\ 1 + E\big(a(i-1),b(j-1)\big) & \text{if } a_i \neq b_j \end{cases} \end{cases}$$

- Recursive definition; we can initially set $i = n$ and $j = m$, using the full strings!

- Thus, to compute $E(a,b)$, we can start with

$$E\big(a(1),\varepsilon\big) = 1 \qquad E\big(\varepsilon,b(1)\big) = 1 \qquad E\big(a(1),b(1)\big) = \begin{cases} 0 & \text{if } a_i = b_j \\ 1 & \text{if } a_i \neq b_j \end{cases}$$

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–							
1	S							
2	A							
3	T							
4	U							
5	R							
6	D							
7	A							
8	Y							

- We can prepare a matrix with $|a| + 1$ rows and $|b| + 1$ columns
- Each row and column is labeled with one character of a and b, respectively
- Exception: first row and column are labeled with "$-$" (=gap/empty)
- Using 0-based indexing, the cell (n, m) is the one in the bottom right corner

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–							
1	S							
2	A							
3	T							
4	U							
5	R							
6	D							
7	A							
8	Y							

- We use this table to compute, for each i and j, the edit distance for the prefixes $a(i)$ and $b(j)$
  - Cell (i, j) will contain the edit distance for $a(i)$ and $b(j)$
  - Row 0 ("$-$"): edit distances between $a(0) = \varepsilon$ and $b(j)$
  - Column 0 ("$-$"): edit distances between $a(i)$ and $b(0) = \varepsilon$
  - Most important: cell (n, m) will contain the value of the edit distance between $a = a(n)$ and $b = b(m)$!

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1						
2	A	2						
3	T	3						
4	U	4						
5	R	5						
6	D	6						
7	A	7						
8	Y	8						

- Row 0 ("–"): edit distances between $a(0) = \varepsilon$ and $b(j)$
  - Since $E(\varepsilon, x) = |x|$, we have $E\big(\varepsilon, b(j)\big) = j$!
- Column 0 ("–"): edit distances between $a(i)$ and $b(0) = \varepsilon$
  - Since $E(x, \varepsilon) = |x|$, we have $E\big(a(i), \varepsilon\big) = i$!

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1						
2	A	2						
3	T	3						
4	U	4						
5	R	5						
6	D	6						
7	A	7						
8	Y	8						

- Let's take a look at cell $(1,1)$, which is supposed to be $E\big(a(1),b(1)\big)$:
  - Remember:

$$E\big(a(i),b(j)\big) = min \begin{cases} 1+E\big(a(i),b(j-1)\big) \\ 1+E\big(a(i-1),b(j)\big) \\ \begin{cases} E\big(a(i-1),b(j-1)\big) & \text{if } a_i = b_j \\ 1+E\big(a(i-1),b(j-1)\big) & \text{if } a_i \neq b_j \end{cases} \end{cases}$$

  - We already have $E(\varepsilon,\varepsilon)$, $E\big(\varepsilon,b(1)\big)$ and $E\big(a(1),\varepsilon\big)$ in cells $(0,0)$, $(0,1)$, $(1,0)$!

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0					
2	A	2						
3	T	3						
4	U	4						
5	R	5						
6	D	6						
7	A	7						
8	Y	8						

- Let's take a look at cell $(1,1)$, which is supposed to be $E\big(a(1),b(1)\big)$:
  - Remember:

$$E\big(a(i),b(j)\big) = min \begin{cases} 1+1 & \text{from cell}(1,0) \\ 1+1 & \text{from cell}(0,1) \\ \begin{cases} 0 & \text{from cell}(0,0) \text{ since } a_i = b_j \end{cases} \end{cases}$$

  - Thus $E\big(a(1),b(1)\big) = 0$ (minimum of 2, 2, and 0)!

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1				
2	A	2	1					
3	T	3						
4	U	4						
5	R	5						
6	D	6						
7	A	7						
8	Y	8						

- Now, we can apply the same rules for cells $(1,2)$ and $(2,1)$!

- For each cell (i,j), take the **minimum** of:
  - Cell to the left + 1 (insertion of a gap in a)
  - Cell to the top + 1 (insertion of a gap in b)
  - If $a(i) = b(j)$: diagonal cell to the top left + 0 (same symbol in both strings)
  - If $a(i) \neq b(j)$: diagonal cell to the top left + 1 (different symbols)

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- For each cell (i, j), take the minimum of:
  - Cell to the left + 1 (insertion of a gap in a)
  - Cell to the top + 1 (insertion of a gap in b)
  - If $a(i) = b(j)$: diagonal cell to the top left + 0 (same symbol in both strings)
  - If $a(i) \neq b(j)$: diagonal cell to the top left + 1 (different symbols)

- We can fill the cells *in any order* (e.g., row by row, column by column)!
  - Let's first complete row 1 ... then row 2 ... then row 3 ... then the rest ... $E(a, b) = 3$!

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- We have computed the edit distance, but *what is the optimal series of operations* transforming one string into the other (i.e., what is the best alignment)?
  - The matrix just tells us that we need 3 operations (substitution, insertion or deletion of a symbol), but what operations do we need to make and where?
- Each time we fill a cell, we look up three neighbors and choose the one which gives the minimum value
  - Idea: keep track of the cells/values we used!

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- Let's start from cell $(8,6)$ (*solution*) to perform "backtracking"
  - For cell $(8,6)$, the three choices where: 4+1 (left), 4+1 (top) and 3+0 (diagonal)
  - For cell $(7,5)$, the three choices where: 4+1 (left), 4+1 (top) and 3+0 (diagonal)
  - For cell $(6,4)$, the three choices where: 4+1 (left), 4+1 (top) and 3+0 (diagonal)
  - For cell $(5,3)$, the three choices where: 3+1 (left), 3+1 (top) and 2+1 (diagonal)
    Important: we moved diagonal and increased the distance by one $\rightarrow$ wrong symbol!
  - For cell $(4,2)$, the three choices where: 3+1 (left), 2+1 (top) and 2+0 (diagonal)
  - For cell $(3,1)$?

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- Let's start from cell $(8,6)$ (*solution*) to perform "backtracking"
  - For cell $(3,1)$, the three choices where: 3+1 (left), 1+1 (top) and 2+1 (diagonal)
    This means, we advanced in string *a* but *not* in string *b* ...
    Hence, we had to insert a gap in string *b*!
  - For cell $(2,1)$, the three choices where: 2+1 (left), 0+1 (top) and 1+1 (diagonal)
    Another gap in string *b*!
  - For cell $(1,1)$, the three choices where: 1+1 (left), 1+1 (top) and 0+0 (diagonal)
    We're back at cell $(0,0)$ (start)

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- How does this translate into an alignment?

S A T U R D A Y
S – – U N D A Y

  - We start at cell (n, m): both strings have Y here!
  - For diagonal movement, we have symbols in both strings: a **match (same edit score)** or a **mismatch/substitution (increased score)**
  - For left or up movement, we have to insert a **gap (symbol in only one string)**

Alignment for minimum edit distance

	b→	0	1	2	3	4	5	6
a↓		–	S	U	N	D	A	Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- To summarize the backtracking:
  - Given strings $a = a_1 a_2 \dots a_n$ (rows) and $b = b_1 b_2 \dots b_m$ (columns):
  - Diagonal move from cell (i, j) to cell $(i-1, j-1)$: align a_i with b_j
  - Vertical move from cell (i, j) to cell $(i-1, j)$: align a_i with $-$ (gap)
  - Horizontal move from cell (i, j) to cell $(i, j-1)$: align b_j with $-$ (gap)

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- A note on terminology:
  - "Gap": insertion of a gap in a or b
  - "Alignment match": a_i aligned with b_j
    - ★ "Sequence match": $a_i = b_j$
    - ★ "Sequence mismatch": a_i and b_j **are aligned(!)**, but $a_i \neq b_j$
  - Often simplified as "match", "mismatch" and "gap"

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- Efficient backtracking:
  - No need to perform all calculations again; we just need to keep track of what choice we made every time we fill a cell in the original computation!
  - Every cell will have a “pointer” to the previous cell which gave it the best solution
  - Note that from every cell there is a path leading to $(0, 0)$!
  - This way, backtracking will just be a walk from cell (n, m) to cell $(0, 0)$, following the pointers

Alignment for minimum edit distance

a↓ \ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- Efficient backtracking:
  - Note: for some cells (e.g., the cell highlighted in light blue) there are multiple neighboring cells which give the same minimum value (here: horizontal and diagonal, both give 2+1=3)
  - The choice is usually arbitrary (multiple paths with the same overall edit distance lead to this cell)
  - In the next example, we will see how this impacts the result

Alignment for minimum edit distance

- What is the time complexity of backtracking (worst case)?
  - The number of backtracking steps is equal to the length of the final alignment (=number of columns in the alignment)

    ```
    S--UNDAY
    |  |.|||
    SATURDAY
    ```

  - Each step takes constant time (follow the pointer; read the needed symbols)
  - In the worst case there is no overlap (common symbol) between the two sequences and the alignment can have a maximum length of $|a| + |b| = n + m$
  - Thus, the time complexity of backtracking is $O(n + m)$, linear in the length of input sequences
  - The overall complexity is determined mostly by the iterative computation of the edit distance, i.e., the construction of the table: $O(nm)$

Alignment for minimum edit distance

a↓ b→		0 –	1 S	2 U	3 N	4 D	5 A	6 Y
0	–	0	1	2	3	4	5	6
1	S	1	0	1	2	3	4	5
2	A	2	1	1	2	3	3	4
3	T	3	2	2	2	3	4	4
4	U	4	3	2	3	3	4	5
5	R	5	4	3	3	4	4	5
6	D	6	5	4	4	3	4	5
7	A	7	6	5	5	4	3	4
8	Y	8	7	6	6	5	4	3

- Partial alignments for prefixes $a(i)$ and $b(j)$: $E\big(a(i), b(j)\big)$
  - Example: partial alignment for $a(2)$="SA" and $b(4)$="SUND"
  - Start at cell $(2, 4)$, highlighted in light blue, and proceed normally:

```
SUND
SA--
```

Alignment for minimum edit distance

	-	S	U	C	R	E
-	0	1	2	3	4	5
S	1	**0**	1	2	3	4
U	2	1	0	1	2	3
G	3	2	1	1	2	3
A	4	3	2	2	2	3
R	5	4	3	3	2	3

- Another example: *a*="SUGAR" and *b*="SUCRE"
  - Edit distance $E(a, b) = 3$
  - We have two possibilities for obtaining 3 in the final cell
  - Hence, we have two possibilities for backtracking!
    - ★ The overall edit distance is always 3, independent of the path we chose!
    - ★ BUT: the two alignments will be different!

Alignment for minimum edit distance

	-	S	U	C	R	E
-	0	1	2	3	4	5
S	1	**0**	1	2	3	4
U	2	1	0	1	2	3
G	3	2	1	1	2	3
A	4	3	2	2	2	3
R	5	4	3	3	2	3

- Another example: *a*="SUGAR" and *b*="SUCRE"
  - Edit distance $E(a, b) = 3$
  - Choice 1: using the diagonal move for the last cell
  - Alignment:

SUGAR
SUCRE

Alignment for minimum edit distance

	-	S	U	C	R	E
-	0	1	2	3	4	5
S	1	**0**	1	2	3	4
U	2	1	0	1	2	3
G	3	2	1	1	2	3
A	4	3	2	2	2	3
R	5	4	3	3	2	3

- Another example: *a*="SUGAR" and *b*="SUCRE"
  - Edit distance $E(a, b) = 3$
  - Choice 2: using the horizontal move for the last cell
  - Alignment:

SUGAR−
SUC−RE

Alignment for minimum edit distance

	-	S	U	C	R	E
-	0	1	2	3	4	5
S	1	**0**	1	2	3	4
U	2	1	0	1	2	3
G	3	2	1	1	2	3
A	4	3	2	2	2	3
R	5	4	3	3	2	3

- Another example: *a*="SUGAR" and *b*="SUCRE"
  - Edit distance $E(a, b) = 3$
  - Choice 3: using the horizontal move for the last cell
  - Alignment:

SUGAR−
SU−CRE

Alignment for minimum edit distance

	-	S	U	C	R	E
-	0	1	2	3	4	5
S	1	0	1	2	3	4
U	2	1	0	1	2	3
G	3	2	1	1	2	3
A	4	3	2	2	2	3
R	5	4	3	3	2	3

	-	S	U	C	R	E
-	0	1	2	3	4	5
S	1	0	1	2	3	4
U	2	1	0	1	2	3
G	3	2	1	1	2	3
A	4	3	2	2	2	3
R	5	4	3	3	2	3

	-	S	U	C	R	E
-	0	1	2	3	4	5
S	1	0	1	2	3	4
U	2	1	0	1	2	3
G	3	2	1	1	2	3
A	4	3	2	2	2	3
R	5	4	3	3	2	3

- Another example: *a*=“SUGAR” and *b*=“SUCRE”
  - Same edit distance, but three different solutions!

```
SUGAR     SUGAR-     SUGAR-
SUCRE     SUC-RE     SU-CRE
```

- From an algorithmic point of view, these solutions are equivalent!
  There is no automatic criterion that can be applied
- Which solution is best depends on the (biological) problem we try to model/solve!
  - We might prefer aligning C with G (solution 2) because they sound similar
  - We might prefer solution 1 in a context where gaps are unlikely and should be penalized
  - ...
- Idea: maybe treating all edit operations as “the same” (equivalent), regardless of the operations and the letters/symbols involved, is not accurate enough?

Alignment for minimum edit distance

"Weighted" edit distance

- Reformulation of the minimization problem:
  Given two strings a and b on an alphabet Σ, the edit distance $E(a,b)$ can also be defined as the minimum-weight series of edit operations that transforms a into b

  - The three operations by Levenshtein in 1966 (insertion, deletion or substitution of a symbol) are the same

  - But: instead of counting "1" for each operation, we can assign a different value ("weight") to each of them!

$$E(a(i),b(j)) = min \begin{cases} \mathbf{w_{id}} + E(a(i),b(j-1)) \\ \mathbf{w_{id}} + E(a(i-1),b(j)) \\ \begin{cases} E(a(i-1),b(j-1)) & \text{if } a_i = b_j \\ \mathbf{w(a_i,b_j)} + E(a(i-1),b(j-1)) & \text{if } a_i \neq b_j \end{cases} \end{cases}$$

  - We can use the same weight w_{id} for insertions and deletions
  - For substitutions the weight $w(a_i,b_j)$ depends on the symbols, hence we can prepare a "weight matrix" for substitutions of symbols from Σ

  - Weights are positive; high weight = strong penalization of the edit operation!

Alignment for minimum edit distance

How to define/determine a weight matrix?

- Example: misspellings in the English language

sub[X, Y] = Substitution of X (incorrect) for Y (correct)

X \ Y (correct)	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	0	0	7	1	342	0	0	2	118	0	1	0	0	3	76	0	0	1	35	9	9	0	1	0	5	0
b	0	0	9	9	2	2	3	1	0	0	0	5	11	5	0	10	0	0	2	1	0	0	8	0	0	0
c	6	5	0	16	0	9	5	0	0	0	1	0	7	9	1	10	2	5	39	40	1	3	7	1	1	0
d	1	10	13	0	12	0	5	5	0	0	2	3	7	3	0	1	0	43	30	22	0	0	4	0	2	0
e	388	0	3	11	0	2	2	0	89	0	0	3	0	5	93	0	0	14	12	6	15	0	1	0	18	0
f	0	15	0	3	1	0	5	2	0	0	0	3	4	1	0	0	0	6	4	12	0	0	2	0	0	0
g	4	1	11	11	9	2	0	0	0	1	1	3	0	0	2	1	3	5	13	21	0	0	1	0	3	0
h	1	8	0	3	0	0	0	0	0	0	2	0	12	14	2	3	0	3	1	11	0	0	2	0	0	0
i	103	0	0	0	146	0	1	0	0	0	0	6	0	0	49	0	0	0	2	1	47	0	2	1	15	0
j	0	1	1	9	0	0	1	0	0	0	0	2	1	0	0	0	0	0	5	0	0	0	0	0	0	0
k	1	2	8	4	1	1	2	5	0	0	0	0	5	0	2	0	0	0	6	0	0	0	4	0	0	3
l	2	10	1	4	0	4	5	6	13	0	1	0	0	14	2	5	0	11	10	2	0	0	0	0	0	0
m	1	3	7	8	0	2	0	6	0	0	4	4	0	180	0	6	0	0	9	15	13	3	2	2	3	0
n	2	7	6	5	3	0	1	19	1	0	4	35	78	0	0	7	0	28	5	7	0	0	1	2	0	2
o	91	1	1	3	116	0	0	0	25	0	2	0	0	0	0	14	0	2	4	14	39	0	0	0	18	0
p	0	11	1	2	0	6	5	0	2	9	0	2	7	6	15	0	0	1	3	6	0	4	1	0	0	0
q	0	0	1	0	0	0	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
r	0	14	0	30	12	2	2	8	2	0	5	8	4	20	1	14	0	0	12	22	4	0	0	1	0	0
s	11	8	27	33	35	4	0	1	0	1	0	27	0	6	1	7	0	14	0	15	0	0	5	3	20	1
t	3	4	9	42	7	5	19	5	0	1	0	14	9	5	5	6	0	11	37	0	0	2	19	0	7	6
u	20	0	0	0	44	0	0	0	64	0	0	0	0	2	43	0	0	4	0	0	0	0	2	0	8	0
v	0	0	7	0	0	3	0	0	0	0	0	1	0	0	1	0	0	0	8	3	0	0	0	0	0	0
w	2	2	1	0	1	0	0	2	0	0	1	0	0	0	0	7	0	6	3	3	1	0	0	0	0	0
x	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0	0	0	0	0	0
y	0	0	2	0	15	0	1	7	15	0	0	0	2	0	6	1	0	7	36	8	5	0	0	1	0	0
z	0	0	0	7	0	0	0	0	0	0	0	7	5	0	0	0	0	2	21	3	0	0	0	0	3	0

- Note: these are not edit weights; these are frequencies with which different substitutions/misspellings can be observed in English language texts
- Edit weights can be computed from observed frequencies
  - Example: “e” $\rightarrow$ “a” is frequent and should have a low edit weight
  - Example: “y” $\rightarrow$ “m” is very rare and should be penalized (high edit weight)
  - In biology: base substitution frequencies in “homologous” sequences during evolution; chemical properties and substitution fequencies of amino acids; ...

Measuring sequence similarity

Measuring similarity instead of distance

- Instead of measuring the distance between strings, evaluate their similarity
- Requires the definition and computation of measures of similarity

"Percent identity"

- After alignment: determine percentage (fraction of positions) of the alignment which does not indicate and edit operation (e.g., no substitution or gap)
  - Example: percent identity = 5/8 = 62.5%

```
S--UNDAY                Problem:      SUGAR   vs.      SUGAR-
|  |.|||                          40% ||...       50% ||. |
SATURDAY                            SUCRE            SUC-RE
```

  - The percent identity may differ between different alignments even when they have the same edit distance!

Measuring sequence similarity

Better: "longest common subsequence"

- Subsequence: derived by deleting *some*, *all* or *no* characters; the order of the remaining elements remains unchanged
  - The remaining symbols need not be consecutive, e.g., "*SUDAY*" is a valid subsequence of "*SAT*U*R*D*AY*"
  - For each string a, both a itself and the empty string (ε) are valid subsequences
  - Longest common subsequence (LCS) problem

- Note the difference between *subsequence* and *substring* (not: longest common substring problem)!
  - Substrings are *consecutive* letters from the original string
  - A string a of length $n = |a|$ has $1 + \sum_{i=1}^{n} i = 1 + \frac{n(n+1)}{2}$ substrings
  - How about the number of subsequences?
  - Idea: a subsequence of a can be described by a binary string B of n elements, telling us for each position i if the corresponding character of a is kept in the subsequence (1) or deleted (0). Example: a=apple, B=11010 $\rightarrow$ subseq=apl
  - Thus, the maximum number of possible subsequences of length n is the same as the number of binary strings of that length: 2^n

The LCS algorithm

- Let a and b be two strings with $|a| = n$ and $|b| = m$
  - Let $a(i)$ and $b(j)$ be prefixes of length $0 \leq i \leq n$ and $0 \leq j \leq m$
- Let $LCS(a, b)$ be the *length* of the longest common subsequence
  - Of course, we have $LCS(a, \varepsilon) = 0$
- We can compute $LCS(a, b)$ using

$$LCS(a(i), b(j)) = \max \begin{cases} \mathbf{0} + LCS(a(i), b(j-1)) \\ \mathbf{0} + LCS(a(i-1), b(j)) \\ \begin{cases} \mathbf{1} + LCS(a(i-1), b(j-1)) & \text{if } a_i = b_j \\ \mathbf{0} + LCS(a(i-1), b(j-1)) & \text{if } a_i \neq b_j \end{cases} \end{cases}$$

This is similar to computing the edit distance $E(a, b)$, but:

- **Maximum** instead of minimum!
- Increase the score by 1 for $a_i = b_j$, no increase for all edit operations (insertion, deletion, change of symbol)
- Thus, LCS will be the maximum number of matching symbols without changing the order (minimum number of gaps and mismatches)
- As for the edit distance, we can use pointers to keep track of the choices

The LCS algorithm

	-	S	U	C	R	E
-	0	0	0	0	0	0
S	0	**1**	1	1	1	1
U	0	1	2	2	2	2
G	0	1	2	2	2	2
A	0	1	2	2	2	2
R	0	1	2	2	**3**	**3**

Example: LCS of “SUGAR” and “SUCRE”

- We set up the same table as for the edit distance, but set the first row and column to 0, because $LCS\big(a(i), \varepsilon\big) = 0 \ \forall i$ and $LCS\big(\varepsilon, b(j)\big) = 0 \ \forall j$
- Fill the cells progessively: cell (i, j) will contain $LCS\big(a(i), b(j)\big)$
  Take the best preceeding cell (horizontal, vertical or diagonal), but adding 1 for a diagonal move with $a_i = b_j$
- We find that the longest common subsequence (LCS) has 3 symbols

The LCS algorithm

	-	S	U	C	R	E
-	0	0	0	0	0	0
S	0	1	1	1	1	1
U	0	1	2	2	2	2
G	0	1	2	2	2	2
A	0	1	2	2	2	2
R	0	1	2	2	3	3

Example: LCS of “SUGAR” and “SUCRE”

- We can trace back the path to rebuild the solution string (LCS)
- We find a character/symbol belonging to the solution every time we made a diagonal move with a score increased by 1 (here: from blue cell to yellow cell)!
  - Increase 2→3 for “R” SUR
  - Increase 1→2 for “U” SUR
  - Increase 0→1 for “S”
- Different paths regard cells with equal score contribution; LCS doesn’t change!

Combining distance and similarity

- We have seen two symmetrical approaches for comparing sequences/strings
  - **Edit distance**: measures the number of differences between two strings, where the edit operations used reproduce what makes sequences different (e.g., difference in nucleotide sequences due to evolution)
  - **Longest common subsequence**: measures the number of characters in the strings which have been "conserved", that is, unchanged from an evolutionary point of view
- Idea: combine the two approaches into a single one!
  Define a similarity measure for two strings *a* and *b* such that
  - **"differences" have a negative effect**: negative weight for gap (insertion, deletion) and mismatch (substitution); often called **"penalty"**
  - **"conserved letters" have a positive effect**: positive weight for match

Combining distance and similarity

We can combine the distance measure $E\big(a(i), b(j)\big)$ with the similarity measure $LCS\big(a(i), b(j)\big)$ to a combined alignment score $A\big(a(i), b(j)\big)$ as follows:

$$A\big(a(i), b(j)\big) = \max \begin{cases} \mathbf{w_d} + A\big(a(i), b(j-1)\big) \\ \mathbf{w_d} + A\big(a(i-1), b(j)\big) \\ \begin{cases} \mathbf{w_m} + A\big(a(i-1), b(j-1)\big) & \text{if } a_i = b_j \\ \mathbf{w_s} + A\big(a(i-1), b(j-1)\big) & \text{if } a_i \neq b_j \end{cases} \end{cases}$$

- $\mathbf{w_d} < 0$: negative **gap penalty** for insertions/deletions
- $\mathbf{w_s} < 0$: negative **mismatch penalty** for substitutions
- $\mathbf{w_m} > 0$: positive **match score** for the same symbol

Goal: find the alignment which **maximizes** this score using dynamic programming

Example:

- $w_s = -1$ (mismatch / substitution)
- $w_d = -2$ (gap / insertion or deletion)
- $w_m = +1$ (match / equal character)

Combining distance and similarity

	-	C	A	A	G	A	C
-	0	-2	-4	-6	-8	-10	-12
G	-2	-1	-3	-5	-5	-7	-9
A	-4	-3	0	-2	-4	-4	-6
A	-6	-5	-2	1	-1	-3	-5
C	-8	-5	-4	-1	0	-2	-2

Example: "CAAGAC" and "GAAC" with $w_s = -1$ (mismatch); $w_d = -2$ (gap); $w_m = +1$

- Prepare a matrix as for edit distance and LCS
  - Set cell $(0,0) = 0$
  - Initialize cells $(i,0)$ of the first column with $i \times w_d$
  - Initialize cells $(0,j)$ of the first row with $j \times w_d$
- Cell $(1,1)$: choices are -2-2=-4 (horizontal), -2-2=-4 (vertical), 0-1=-1 (diagonal)
- Final alignment score: -2
- Traceback; final alignment:

CAAGAC
GAA--C

(Note: some possible traceback pointers are missing! There are other solutions; can you find them?)

From alignment to score

- If we know the weights used when applying the algorithm (here: $w_d = -2$, $w_s = -1$ and $w_m = +1$), we can determine the final score of an alignment directly from the alignment itself
  - Each column of the alignment represents a match or an edit operation
  - Associate with each column its score and sum over all individual scores:

```
 C   A   A   G   A   C
 G   A   A   -   -   C
-1  +1  +1  -2  -2  +1 = -2
```

Biological sequence alignment

- The algorithm we have just developed is not restricted to biological sequence alignment, but very frequently used for this purpose!

- Introduced in 1970 by Saul B. Needleman and Christian D. Wunsch ("Needleman-Wunsch algorithm"; Needleman & Wunsch, 1970)
  - Initially introduced for protein (amino acid) sequences

- The algorithm is used for (and sometimes also called) "global sequence alignment"
  - As opposed to "local sequence alignment", which we will see shortly!

- Once finished, we keep only the alignment and the alignment score (the table can be discarded)

- Online version for testing:
  `https://rna.informatik.uni-freiburg.de/Teaching/index.jsp?toolName=Needleman-Wunsch`

Example: protein sequences

Alignment example, split over multiple lines:

```
SSH_UOMO        -MLLLARCLLLVLVSSLLVCSGLACGPGRGFGKRRHPKKLTPLAYKQFIPNVAEKTLGAS
SSH_TOPO        MLLLLARCFLVILASSLLVCPGLACGPGRGFGKRRHPKKLTPLAYKQFIPNVAEKTLGAS
                 :******:*::*.******.************************************

SSH_UOMO        GRYEGKISRNSERFKELTPNYNPDIIFKDEENTGADRLMTQRCKDKLNALAISVMNQWPG
SSH_TOPO        GRYEGKITRNSERFKELTPNYNPDIIFKDEENTGADRLMTQRCKDKLNALAISVMNQWPG
                *******:****************************************************

SSH_UOMO        VKLRVTEGWDEDGHHSEESLHYEGRAVDITTSDRDRSKYGMLARLAVEAGFDWVYYESKA
SSH_TOPO        VKLRVTEGWDEDGHHSEESLHYEGRAVDITTSDRDRSKYGMLARLAVEAGFDWVYYESKA
                ************************************************************

SSH_UOMO        HIHCSVKAENSVAAKSGGCFPGSATVHLEQGGTKLVKDLSPGDRVLAADDQGRLLYSDFL
SSH_TOPO        HIHCSVKAENSVAAKSGGCFPGSATVHLEQGGTKLVKDLRPGDRVLAADDQGRLLYSDFL
                ************************************ ********************

SSH_UOMO        TFLDRDDGAKKVFYVIETREPRERLLLTAAHLLFVAPHNDSATGEPEASSGSGPPSGGAL
SSH_TOPO        TFLDRDEGAKKVFYVIETLEPRERLLLTAAHLLFVAPHND----------SGPTPG---
                ******:*********** ********************          ***..*

SSH_UOMO        GPRALFASRVRPGQRVYVVAERDGDRRLLPAAVHSVTLSEEAAGAYAPLTAQGTILINRV
SSH_TOPO        -PSALFASRVRPGQRVYVVAERGGDRRLLPAAVHSVTLREEEAGAYAPLTAHGTILINRV
                 * ******************.*************** ** *********:********

SSH_UOMO        LASCYAVIEEHSWAHRAFAPFRLAHALLAALAPARTDRGGDSGGGDRGGGGGRVALTAPG
SSH_TOPO        LASCYAVIEEHSWAHRAFAPFRLAHALLAALAPARTD----------GGGGGSIP-AAQS
                ************************************          ***** :. :* .

SSH_UOMO        AADAPGAGATAGIHWYSQLLYQIGTWLLDSEALHPLGMAVKSS
SSH_TOPO        ATEARGAEPTAGIHWYSQLLYHIGTWLLDSETMHPLGMAVKSS
                *::* ** .************:*********::**********
```

Proteins encoded by the human *SHH* gene (top) and the mouse *SHH* gene (bottom)

SSH = sonic hedgehog signaling molecule

Example: protein sequences

Another alignment example, split over multiple lines:

Range 1: 1 to 136 GenPept Graphics ▼ Next Match ▲ Previous Match

Score	Expect	Method	Identities	Positives	Gaps
248 bits(633)	3e-87	Compositional matrix adjust.	121/136(89%)	130/136(95%)	0/136(0%)

```
Query   1     MARTKQTARKSTGGKAPRKQLATKAARKSAPATGGVKKPHRYRPGTVALREIRRYQKSTE   60
              MARTKQTARKSTGGKAPRKQLA+KAARKSAP+TGGVKKPHRY+PGTVALREIRR+QKSTE
Sbjct   1     MARTKQTARKSTGGKAPRKQLASKAARKSAPSTGGVKKPHRYKPGTVALREIRRFQKSTE   60

Query   61    LLIRKLPFQRLVREIAQDFKTDLRFQSSAVMALQEACEAYLVGLFEDTNLCAIHAKRVTI   120
              LLIRKLPFQRLVREIAQDFKTDLRFQSSA+ ALQE+ EAYLV LFEDTNL AIHAKRVTI
Sbjct   61    LLIRKLPFQRLVREIAQDFKTDLRFQSSAIGALQESVEAYLVSLFEDTNLAAIHAKRVTI   120

Query   121   MPKDIQLARRIRGERA  136
                KDI+LARR+RGER+
Sbjct   121   QKKDIKLARRLRGERS  136
```

Proteins encoded by the human histone 3 (*H3*) gene (top) and the homologuous gene of *S. cerevisiae* (yeast; bottom)

A note on gap penalties

- There are actually different models of gap penalties:
  - Linear gap penalty: gap penalty for a single symbol is multiplied by the total length of a gap (this is what we implicitly used so far), e.g., "$---$" counts the same as $3\times$"$-$"
  - Constant gap penalty: fixed negative score for the entire gap, regardless of its length (also when the gap covers multiple symbols of the other sequence), e.g., "$---$" counts the same as an individual "$-$"
    - ⋆ Used if fewer, larger, gaps are preferred, thus leaving larger contiguous sections
  - Affine gap penalty (most widely used): combines the constant and linear gap penalty by defining:
    1. Constant *gap opening penalty*: penalty for opening a gap (first "$-$")
    2. Linear *gap extension penalty*: penalty for extending the length of an existing gap by 1 "$-$"
    - ⋆ Usually: gap opening penalty $>$ gap extension penalty; i.e., it's easier to extend an existing gap then to open a new one
    - ⋆ Purpose: prefer not to have gaps, but if they are necessary prefer fewer large ones
  - ...[4]

[4] See, for example, `https://en.wikipedia.org/wiki/Gap_penalty` for more detailed descriptions and additional approaches.

References I



Needleman S.B. & Wunsch C.D. (1970)
A general method applicable to the search for similarities in the amino acid sequence of two proteins
Journal of Molecular Biology 48(3):443–453